SOFTWARE ENGINEERING (24MX21)

SOFTWARE REQUIREMENT SPECIFICATION DOCUMENT

PSG TECH HOSTELS - Cross-Platform Smart Hostel Management System

ARAVINDH KANNAN M S (25MX304)

DAYANANDA J (25MX308)

MASTER OF COMPUTER APPLICATIONS

ANNA UNIVERSITY

APRIL 2026

DEPARTMENT OF COMPUTER APPLCATIONS

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE - 641014

Date: 06-04-2026

Version: 0.4

TABLE OF CONTENTS

3. **External interface requirements**
    3.1. User interface requirements
    3.2. Hardware interface requirements
    3.3. Software interface requirements
    3.4. Communication interface requirements
4. **Non-functional requirements**
    4.1. Security
    4.2. Capacity
    4.3. Compatibility
    4.4. Reliability
    4.5. Scalability
    4.6. Maintainability
    4.7. Usability
    4.8. Portability
5. **Definitions and acronyms**
6. **Use Case Model**
7. **Use Case Story**

Software Requirements Specification

PSG TECH HOSTELS SYSTEM

| **Project Name:** PSG TECH HOSTELS - Cross-Platform Smart Hostel Management System

| **Date:** April 05, 2026

| **Version:** 0.4

| **By:** Aravindh Kannan M S, Dayananda J

Revision History			
Date	Version	Author	Description
March 01, 2026	0.1	Aravindh Kannan M S, Dayananda J	Initial Draft
March 15, 2026	0.2	Aravindh Kannan M S, Dayananda J	Student, Warden Module Structured
March 22, 2026	0.3	Aravindh Kannan M S, Dayananda J	Admin Module Defined and Database Integration

Review History		
Date	Reviewer	Comments
February 26, 2026	Dr. V. Umarani	
March 26, 2026(Review 1)	Dr. V. Umarani, Ms. R. Aruna	

1. Introduction

This document describes the requirements for a unified cross-platform mobile and web application designed to modernize the hostel management operations at PSG College of Technology.

1.1 Product Scope

- The project aims to provide a unified platform for digital service delivery.
- It focuses on high availability and seamless integration between web and mobile interfaces.
- The primary goal is to enhance user engagement through a responsive and intuitive design.

1.2 Product Value

- The system provides a lightweight, responsive alternative to the current desktop-oriented platform.
- Introducing real-time notifications for leave approvals and emergencies, ensuring data is instantly reflected across all devices.

1.3 Intended Audience

The product is intended to serve:

- Students
- Faculty Guides
- Hostel Administrators
- PSG TECH Management

1.4 Intended Use

- **Students:** Apply for leave digitally, track approval status, raise maintenance complaints, and view the digital notice board.
- **Wardens:** Approve or reject leave requests, update complaint resolution status, and view room details.
- **Admin:** Allocate rooms, manage user records, broadcast notices, and generate reports.

1.5 General Description

The software will perform core hostel functions including student management, room allocation, leave tracking, and complaint handling. Key features include Firebase Authentication for secure login, Cloud Firestore for real-time data, and Firebase Cloud Messaging for instant push notifications.

2. Functional Requirements

2.1. Authentication: Secure login using Firebase Authentication with role-based verification.

2.2. Role Based Access: Modules and data access based on Roles.

2.3. Leave Management: Digital leave application for students and an approval/rejection interface for wardens.

2.4. Complaint Tracking: Ability for students to submit complaints and for wardens to update resolution status.

2.5. Administrative Tools: Room allocation, user record management, and digital notice broadcasting.

2.6. Notifications and Data Integrity: Automatic push notifications for leave status updates and emergency alerts and Real-time synchronization of all data across web and mobile platforms via Cloud Firestore.

2.7. Mess Token Application: Allows students to apply for mess food tokens.

2.8. Display of Mess Bill: Mess Bill Reductions and Monthly Food Bill.

3. External Interface Requirements

3.1 User Interface Requirements

- The UI must be responsive and intuitive, optimized for both handheld mobile devices and desktop browsers.
- It should follow mobile-first design principles to address the lack of responsiveness in the existing system.

3.2 Hardware Interface Requirements

- **Mobile:** Supported on Android and iOS devices.
- **Network:** Requires standard internet connectivity for cloud synchronization and push notifications.

3.3 Software Interface Requirements

- **Frontend:** Built using modern frameworks (e.g., React/Flutter) as described in the tech stack.
- **Authentication:** Firebase Authentication — handles email/password sign-in, session management via authStateChanges stream, and password reset.
- **Backend:** Integration with a centralized RESTful API and cloud-based database services.

3.4 Communication Interface Requirements

- All communication between the Flutter client and Firebase services occurs over HTTPS using the official Firebase Flutter SDKs.
- Embedded forms for user feedback and support inquiries

4. Non-Functional Requirements

4.1. Security

- All user authentication is handled exclusively through Firebase Authentication. No plaintext passwords are stored in Firestore.
- All data transmission between the application and Firebase services is encrypted over TLS via HTTPS.

4.2. Capacity

- Designed to handle up to 10,000 concurrent users with scalable cloud storage.

4.3. Compatibility

- **Web:** Support for the latest versions of Chrome, Firefox, and Safari.
- **Mobile:** Android 10+ and iOS 14+.

4.4. Reliability

- Target uptime of 99.9% with automated failover mechanisms.

4.5. Scalability

- The architecture must allow for horizontal scaling of microservices during peak loads - Cloud Firestore's horizontally scalable architecture supports increased document volume and concurrent users without backend restructuring.

4.6. Maintainability

- Implementation of Continuous Integration/Continuous Deployment (CI/CD) pipelines for rapid updates.
- Provider-based state management decouples business logic from UI, allowing controllers to be updated or replaced without rewriting screens.

4.7. Usability

- All primary student actions (apply leave, book token, submit complaint, apply for mess) are accessible from the student bottom navigation bar within two taps from the home screen.
- Intuitive navigation with a maximum of three clicks to reach primary functions.

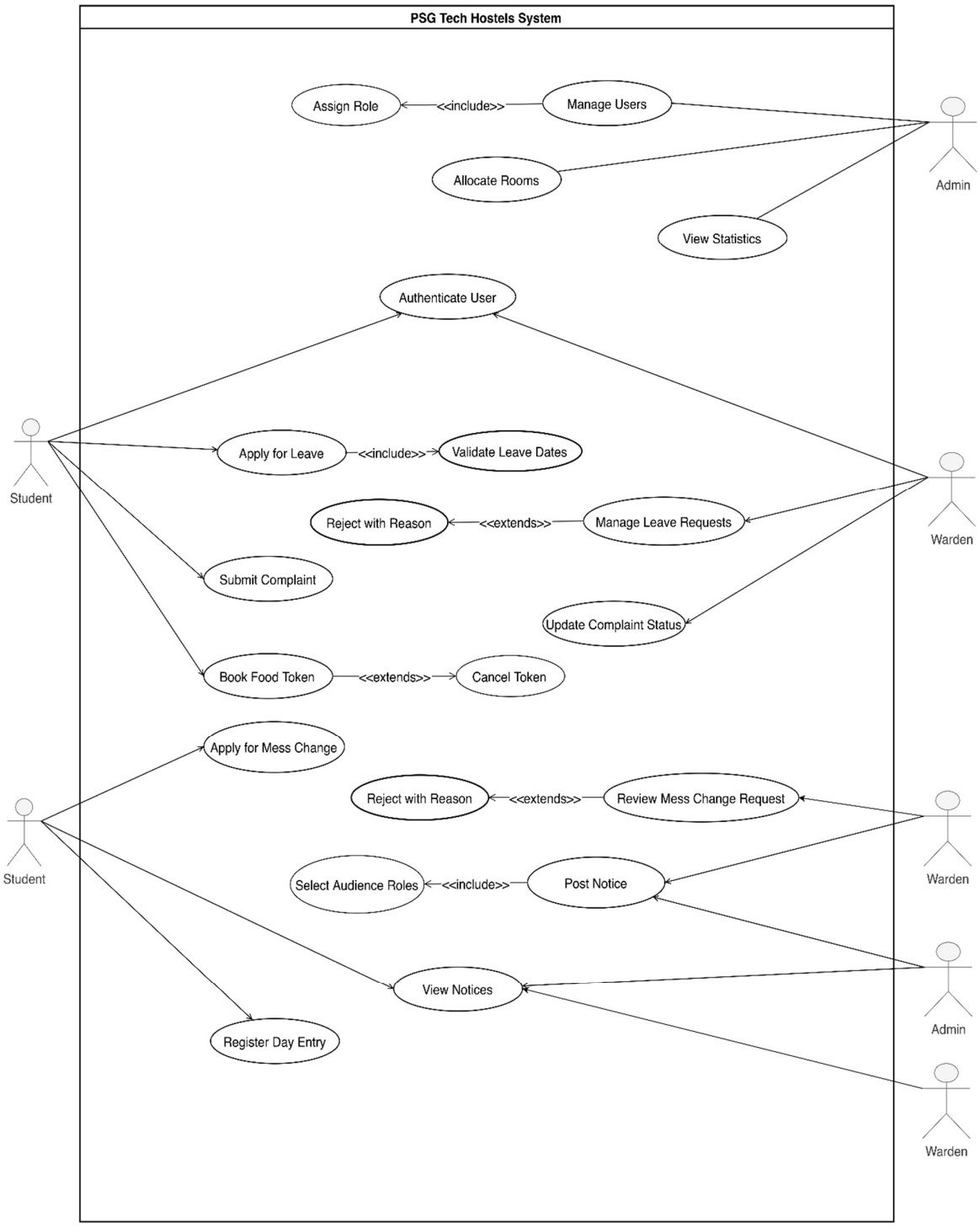
4.8. Portability

- The single Flutter codebase is configured for Android, iOS, macOS, and Web, avoiding platform-specific code duplication in the application layer.

5. Definitions and Acronyms

- **SRS** - Software Requirements Specification
- **MAD** - Mobile Application Development
- **FCM** - Firebase Cloud Messaging
- **Firestore** - Google Firebase's NoSQL real-time cloud Database
- **DART** - Programming language used by Flutter Framework
- **API** - Application Programming Interface
- **HTTPS** - HyperText Transfer Protocol Secure
- **TLS** - Transport Layer Security
- **Provider** - Flutter state management package using ChangeNotifier and Consumer widgets.
- **go_router** - Flutter navigation package used for declarative, role-guarded routing

6. Use-Case Diagram



7. Use-Case Story

USE CASE 1: AUTHENTICATE USER

1. Name of the Use case	Authentication
2. Actors	Student, Warden, Admin
3. Description	This use case describes how users of all three roles securely log into the PSG Tech Hostels application using role-appropriate credentials and are routed to their respective dashboards upon successful authentication.
4. Preconditions	1. The PSG Tech Hostels application is installed and launched on the device. 2. The user has a registered account present in the users Firestore collection. 3. The device has an active internet connection.
5. Triggers	When the application is launched and the Firebase authStateChanges stream detects no active session, the splash screen routes the user to the login screen.
6. Flow of Events 6.1 Basic flow of events	Display Login Screen: The system presents the login screen with three role tabs — Student, Warden, and Admin. 1. Select Role Tab: The user taps the tab matching their role. 2. Enter Credentials: Student enters Roll Number and Password. Warden and Admin enter Email Address and Password. 3. Submit: The user taps Sign In. The

	system passes the credentials to Firebase Authentication. 4. Resolve Role: On successful sign-in, the system reads the role field from the users Firestore document via a real-time snapshot stream. 5. Route to Dashboard: The application redirects the user to the Student, Warden, or Admin dashboard based on the resolved role.
6.2 Alternate flow of events	1. Invalid Credentials: Firebase Authentication returns an error. The system displays an inline error message. The user must re-enter their credentials. 2. Roll Number Format: If the student enters only their roll number without a domain, the system automatically appends @psgtech.ac.in before submitting to Firebase Authentication. 3. Forgot Password: The user taps Forgot Password and enters their registered roll number or email. The system triggers Firebase Authentication's password reset email flow. 4. Unauthorized Route Access: If a user attempts to access a route outside their role prefix, the central route guard blocks access and returns the user to their assigned home route.

7. Success Criteria	The use case concludes when the user is authenticated, their role is resolved from Firestore, and they are routed to their role-specific shell dashboard.
8. Post Conditions	1. Active Session: Firebase maintains the authentication session.The system streams the users document for the duration of the session. 2. Role Enforced: The resolved role is held in memory and enforced by the router for all subsequent navigation.
9. Use Case Extensions	Apply for Leave, Submit Complaint, Book Food Token, Apply for Mess Change, Register Day Entry, View Notices, Manage Leave Requests, Update Complaint Status, Post Notice, Manage Users, Allocate Rooms, View Statistics.

USE CASE 2: APPLY FOR LEAVE

1. Name of the Use case	Apply for Leave
2. Actors	Student
3. Description	This use case describes how a student submits a digital leave request by providing dates, leave type, and reason, and tracks the real-time approval status of all submitted requests.
4. Preconditions	1. The student must be authenticated with role Student. 2. The device must have an active internet connection.
5. Triggers	When the student taps Leave in the bottom navigation bar of the Student shell layout, opening the Leave Request screen.
6. Flow of Events 6.1 Basic flow of events	1. View Leave Screen: The system displays the leave request screen. A real-time Firestore stream populates the student's leave history filtered by their user ID. 2. Open Application Form: The student taps the Apply for Leave button. The leave application form is displayed. 3. Enter Dates: The student selects a start date and end date using the date picker fields. 4. Select Leave Type and Reason: The student selects a leave type from the dropdown and enters a description

	in the reason field. 5. Submit: The student taps Submit. The system passes the form data to the leave request controller. 6. Record Written to Firestore: A new document is created in the leave_requests collection with the fields userId, startDate, endDate, leaveType, reason, status set to Pending, and a createdAt timestamp. 7. History Updated: The real-time Firestore listener refreshes the leave history list to include the new pending request immediately.
6.2 Alternate flow of events	1. End Date Before Start Date: The controller validates that the end date is not earlier than the start date. If it is, an error message is displayed and the form remains open. 2. Missing Required Dates: If either date field is not selected, the controller blocks submission and displays a validation error. 3. Cancel: If the student navigates away without submitting, no document is written and the leave history remains unchanged.
7. Success Criteria	The use case concludes when a leave request document is successfully created in Firestore with status Pending and appears in the student's leave history list.
8. Post Conditions	1. Leave Record Created: A new

	document exists in the leave_requests collection with status Pending. 2. Warden Notified in Real Time: The Manage Leave Requests screen on the warden's device reflects the new request via its own Firestore real-time listener.
9. Use Case Extensions	Manage Leave Requests, Reject with Reason.

USE CASE 3: MANAGE LEAVE REQUESTS

1. Name of the Use case	Manage Leave Requests
2. Actors	Warden
3. Description	This use case describes how a warden reviews all student leave requests, filters them by approval status, and approves or rejects individual requests with an optional rejection reason.
4. Preconditions	1. The warden must be authenticated with role Warden. 2. At least one document must exist in the leave requests Firestore collection.
5. Triggers	When the warden taps Leaves in the bottom navigation bar of the Warden shell layout, opening the Warden Leave Requests screen.
6. Flow of Events 6.1 Basic flow of events	1. Load Leave Requests: The system opens a real-time Firestore stream on the leave requests collection ordered by creation date descending. All documents are loaded and displayed. 2. Filter by Status Tab: The warden selects from four tabs — All, Pending, Approved, or Rejected. The system filters the displayed list client-side to match the selected status. 3. Select Request: The warden taps a specific request. The system displays full details including student name, start date, end date, leave type, and reason.

	4. Approve: The warden taps Approve. The system updates the leave_requests document with status set to Approved, an approvedAt timestamp, and approvedBy set to the warden's user ID. 5. List Refreshed: The real-time Firestore listener reflects the updated status across tabs immediately.
6.2 Alternate flow of events	1. Reject with Reason: The warden taps Reject. A dialog appears with an optional rejection reason text field. The warden confirms. The system updates the document with status set to Rejected, the rejectionReason, a rejectedAt timestamp, and rejectedBy set to the warden's user ID. 2. Empty Pending Tab: If no documents with status Pending exist, the Pending tab displays an empty state and no action is required.
7. Success Criteria	The use case concludes when the leave request document in Firestore is updated with the new status, the corresponding timestamp, and the identity of the approving or rejecting warden.
8. Post Conditions	1. Document Updated: The leave_requests Firestore document reflects the new status, the warden's identity, and the corresponding timestamp. 2. Student View Updated: The student's Leave Request screen reflects the

	updated approval status in real time via its own Firestore listener.
9. Use Case Extensions	Reject with Reason, View Notices.

USE CASE 4: SUBMIT COMPLAINT

1. Name of the Use case	Submit Complaint
2. Actors	Student
3. Description	This use case describes how a student raises a maintenance or hostel-related complaint by providing a title and description, and monitors the resolution status of all submitted complaints in real time.
4. Preconditions	1. The student must be authenticated with role Student. 2. The device must have an active internet connection.
5. Triggers	When the student navigates to the complaint section and taps Submit Complaint, opening the Complaint Submission screen.
6. Flow of Events 6.1 Basic flow of events	1. View Complaints Screen: The system displays the complaint submission screen. A real-time Firestore stream on the complaints collection, ordered by creation date descending, populates the student's complaint history. 2. Open Complaint Form: The student taps Submit Complaint. The form fields Title and Description are presented. 3. Enter Details: The student enters a

	title and a description of the issue. 4. Submit: The student taps Submit. The system passes the data to the complaint controller. 5. Record Written to Firestore: A new document is created in the complaints collection with the fields userId, title, description, status set to Pending, and a createdAt timestamp. 6. Complaints List Updated: The real-time Firestore listener adds the new complaint to the list immediately.
6.2 Alternate flow of events	1. Empty Title or Description: If either field is empty, the complaint controller blocks submission and displays a validation error message. The form remains open. 2. Cancel: If the student exits the form without submitting, no document is written.
7. Success Criteria	The use case concludes when a complaints document is successfully created in Firestore with status Pending and appears in the student's complaint history list.
8. Post Conditions	1. Complaint Record Created: A new document exists in the complaints collection with status Pending. 2. Warden View Updated: The Update Complaint Status screen on the warden's device reflects the new complaint in real time via its own Firestore listener.
9. Use Case Extensions	Update Complaint Status

USE CASE 5: POST NOTICE

1. Name of the Use case	Post Notice
2. Actors	Warden, Admin
3. Description	This use case describes how a warden or administrator creates and publishes a notice to targeted user groups, which becomes immediately visible in the notice feeds of all relevant users in real time.
4. Preconditions	1. The actor must be authenticated with role Warden or Admin. 2. The device must have an active internet connection.
5. Triggers	When the warden taps Notices in the Warden shell bottom navigation bar, or when the admin navigates to the Notice section in the Admin shell, opening the respective Notice posting screen.
6. Flow of Events 6.1 Basic flow of events	1. View Notices Screen: The system displays the notice posting screen and loads all existing active notices via a real-time Firestore stream on the notices collection filtered by isActive set to true, ordered by creation date descending. 2. Open Post Form: The actor taps the Post Notice button. The notice form is displayed. 3. Enter Title and Body: The actor enters a title and body text for the notice. Select Audience Roles — Admin Only (includes): If the actor is

	Admin, the system additionally displays audience checkboxes — Student, Warden, and Admin. The admin selects one or more target audiences. 4. Submit: The actor taps Post. The system calls Firestore add on the notices collection. 5. Document Written: A new document is created in notices with the fields title, body, createdBy, createdByRole, isActive set to true, audienceRoles, and a createdAt timestamp. 6. Notice Feed Updated: The real-time Firestore listener on all connected notice screens refreshes the list immediately.
6.2 Alternate flow of events	1. Empty Title or Body: If either the title or body field is empty, the system displays an inline validation error and blocks the Firestore write operation. 2. No Audience Selected — Admin: If the admin submits without selecting at least one audience role, the system displays a validation error and blocks submission. 3. Delete Notice — Admin Only: The admin taps Delete on an existing notice. The system removes the notices document from Firestore. The notice disappears from all audience

	feeds in real time.
7. Success Criteria	The use case concludes when a notices document is successfully created in Firestore with isActive set to true and the notice appears in the targeted users' notice feeds.
8. Post Conditions	1. Notice Document Created: A new document exists in the notices collection with isActive set to true. 2. All Targeted Users See Notice: The View Notices screens for all targeted role groups reflect the new notice immediately via their respective Firestore real-time listeners.
9. Use Case Extensions	View Notices, Select Audience Roles.

Summary

This Software Requirements Specification (SRS) document outlines the requirements for the PSG TECH HOSTELS - Cross-Platform Smart Hostel Management System. It covers the introduction, functional requirements, and external interface requirements. Further sections, including system features, non-functional requirements, constraints, and future enhancements, are reserved for more details to capture the full scope of the project and offer potential evolution beyond the current specifications.